

NumPy Creating Arrays

[< Previous](#)[Next >](#)

Create a NumPy ndarray Object

NumPy is used to work with arrays. The array object in NumPy is called **ndarray**.

We can create a NumPy **ndarray** object by using the **array()** function.

Example

[Get your own Python Server](#)

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

[Try it Yourself »](#)

type(): This built-in Python function tells us the type of the object passed to it. Like in above code it shows that `arr` is `numpy.ndarray` type.

To create an `ndarray`, we can pass a list, tuple or any array-like object into the `array()` method, and it will be converted into an `ndarray`:

Example

Use a tuple to create a NumPy array:

```
import numpy as np

arr = np.array((1, 2, 3, 4, 5))

print(arr)
```

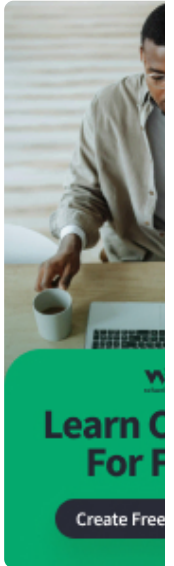
Try it Yourself »

Dimensions in Arrays

A dimension in arrays is one level of array depth (nested arrays).

nested array: are arrays that have arrays as their elements.

ADVERTISEMENT



COLOR
PICKER



[REMOVE ADS](#)

0-D Arrays

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

Example

Create a 0-D array with value 42

```
import numpy as np

arr = np.array(42)

print(arr)
```

[Try it Yourself »](#)

1-D Arrays

An array that has 0-D arrays as its elements is called uni-dimensional or 1-D array.

These are the most common and basic arrays.

Example

Create a 1-D array containing the values 1,2,3,4,5:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)
```

[Try it Yourself »](#)

2-D Arrays

An array that has 1-D arrays as its elements is called a 2-D array.

These are often used to represent matrix or 2nd order tensors.

NumPy has a whole sub module dedicated towards matrix operations called `numpy.mat`

Example

Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])
```

[Tutorials ▼](#)[References ▼](#)[Exercises ▼](#)[Certificates ▼](#)[In](#)[HTML](#) [CSS](#) [JAVASCRIPT](#) [SQL](#) [PYTHON](#) [JAVA](#) [PHP](#) [HOW TO](#) [W3.CSS](#) [C](#) [C](#) [»](#)

NumPy Tutorial

[NumPy HOME](#)[NumPy Intro](#)[NumPy Getting Started](#)[NumPy Creating Arrays](#)[NumPy Array Indexing](#)[NumPy Array Slicing](#)[NumPy Data Types](#)[NumPy Copy vs View](#)[NumPy Array Shape](#)

3-D arrays

An array that has 2-D arrays (matrices) as its elements is called 3-D array.

These are often used to represent a 3rd order tensor.

NumPy Array Reshape
NumPy Array Iterating
NumPy Array Join
NumPy Array Split
NumPy Array Search
NumPy Array Sort
NumPy Array Filter

NumPy Random

Random Intro
Data Distribution
Random Permutation
Seaborn Module
Normal Distribution
Binomial Distribution
Poisson Distribution
Uniform Distribution
Logistic Distribution
Multinomial Distribution
Exponential Distribution
Chi Square Distribution
Rayleigh Distribution
Pareto Distribution

Example

Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]],
                [[1, 2, 3], [4, 5, 6]]])

print(arr)
```

[Try it Yourself »](#)

Check Number of Dimensions?

NumPy Arrays provides the `ndim` attribute that returns an integer that tells us how many dimensions the array have.

Example

Check how many dimensions the arrays have:

```
import numpy as np

a = np.array(42)
b = np.array([1, 2, 3, 4, 5])
c = np.array([[1, 2, 3], [4, 5, 6]])
d = np.array([[[1, 2, 3], [4, 5, 6]],
                [[1, 2, 3], [4, 5, 6]]])

print(a.ndim)
print(b.ndim)
print(c.ndim)
print(d.ndim)
```

Try it Yourself »

Higher Dimensional Arrays

An array can have any number of dimensions.

When the array is created, you can define the number of dimensions by using the `ndmin` argument.

Example

Create an array with 5 dimensions and verify that it has 5 dimensions:

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('number of dimensions :', arr.ndim)
```

Try it Yourself »

In this array the innermost dimension (5th dim) has 4 elements, the 4th dim has 1 element that is the vector, the 3rd dim has 1 element that is the matrix with the vector, the 2nd dim has 1 element that is 3D array and 1st dim has 1 element that is a 4D array.

Exercise ?

Consider the following array:

```
arr = np.array([[1, 2, 3], [4, 5, 6]])
```